

Практика 2. Рекурсия в λ -исчислении

осень 2026

Андрей Стоян, Илья Колегов

осень 2026

Летучка 2

Летучка 2 · 7 минут · Ваше ФИ: _____

1. Функция двух аргументов записана как функция одного аргумента, возвращающая функцию. Как называется этот приём? _____
2. Заполните пропуски в умножении из домашки: **mult** $:= \lambda n\ m. \mathbf{natElim}\ n$ _____
3. Сделайте один β -шаг: $(\lambda x. \lambda y. x)\ y \rightsquigarrow$ _____
4. Python вычисляет аргументы до вызова функции. Как корректно реализовать собственную абстракцию над **if**? Реализуйте рекурсивный факториал.

```
def myif(c, t, e): return _____  
def fac(n): return myif(n <= 1, _____, _____)
```

Условное выражение как функция

Эквивалентен ли код?

```
if b:
    return f()
else:
    return g()
```

```
def myif(cond, t, e):
    if cond:
        return t
    else:
        return e
myif(b, f(), g())
```

Нет: Python вычисляет аргументы до вызова, поэтому `fac` через `myif` не завершается.

```
def fac(n): return myif(n <= 1, 1, n * fac(n - 1))

fac(3) -> myif(3 <= 1, 1, 3 * fac(2))
      -> myif(3 <= 1, 1, 3 * myif(2 <= 1, 1, 2 * fac(1)))
      -> myif(3 <= 1, 1, 3 * myif(2 <= 1, 1, 2 * myif(1 <= 1, 1, 1 * fac(0))))
      ...
```

Перейдём к ленивой семантике вручную — с помощью абстракций:

```
def fac(n):
    return myif(n <= 1,
                lambda: 1,
                lambda: n * fac(n - 1)
    )
```

```
def myif(cond, t, e):
    if cond:
        return t()
    else:
        return e()
```

Ещё метаинформация о курсе

Тutorial по прохождению

- **Перед парой перечитайте лекцию** — всего 15 минут, но очень поможет нам на практиках
- **Управляйте временем и ресурсами**
- **Рефлексируйте над неудачами** — спокойно, работая с причинами: не «я не могу», а «чего не хватило: времени, определения, примера»
- **Застряли на 20 минут — спрашивайте** — преподавателей, одногруппников, в чате; молчаливое непонимание грустно, а вопрос одного экономит время десятерым
- **Спите и отдыхайте** — на дистанции семестра это не роскошь, а производительность

Полезная [памятка](#) от кураторов Computer Science Center.

Цикл без рекурсии

Факториал без рекурсии

natElim n s z повторяет шаг s над z ровно n раз: при известном числе итераций рекурсия не нужна.

```
def fac(n):  
    i, acc = 1, 1  
    for _ in range(n):  
        i, acc = i + 1, i * acc  
    return acc
```

✎ Закодируйте **fac** через **natElim**. Цикл выше держит две переменные, а **natElim** протаскивает через шаги один терм. Что это за терм и каким будет шаг?

⚡ **Решение.**

Состояние — пара «номер шага, накопленное произведение»:

start := **pair** 1 1

step := $\lambda p.$ **pair** $\underbrace{(\text{succ } (\text{fst } p))}_{\text{номер шага}}$ $\underbrace{(\text{mult } (\text{fst } p) (\text{snd } p))}_{\text{произведение}}$

fac := $\lambda n.$ **snd** (**natElim** n **step** **start**)

Выбираем правильный уровень абстракции для рассуждений

fac 3

→ **snd** (($\lambda s z. s (s (s z))$) ($\lambda p. \mathbf{pair} (\mathbf{suc} (\mathbf{fst} p)) (\mathbf{mult} (\mathbf{fst} p) (\mathbf{snd} p))$) (**pair 1 1**))
→ ($\lambda q. q \mathbf{K}_*$) (($\lambda p. \mathbf{pair} (\mathbf{suc} (\mathbf{fst} p)) \dots$) (($\lambda p. \dots$) (($\lambda p. \dots$) (($\lambda x y f. f x y$) **1 1**))))
→ ... и ещё десяток строк, каждая длиннее предыдущей. Да ну серьёзно??

Раскрывать до λ незачем: законы **natElim** и пар уже доказаны, это «большие шаги»:

fac 3

$\rightsquigarrow_{\mathbf{fac}}$ **snd (natElim 3 step start)**
→ **snd (step (step (step start)))** законы **natElim**
→ **snd (step (step (pair 2 1)))** законы пар, **mult 1 1 → 1**
→ **snd (step (pair 3 2)) → snd (pair 4 6) → 6**

Рекурсия и комбинаторы неподвижной точки

Рекурсия

- Одного цикла с фиксированным числом итераций недостаточно, чтобы уметь всё
- Чтобы добиться вычислительной полноты, нужна полноценная рекурсия
- В прикладные языки рекурсия встроена — достаточно в теле именованной функции вызвать её саму
- Но у нас в языке нет именованных функций, же?!

С лекции: рекурсия как неподвижная точка

1. Хотим терм **fac**, удовлетворяющий рекурсивному уравнению:

$$\mathbf{fac} \ n =_{\beta} \mathbf{if} \ (\mathbf{isZero} \ n) \ \mathbf{1} \ (\mathbf{mult} \ n \ (\boxed{\mathbf{fac}} \ (\mathbf{pred} \ n)))$$

2. Абстрагируем правую часть по имени и по аргументу:

$$\mathbf{fac} \ n =_{\beta} \underbrace{(\lambda \boxed{r} \boxed{n}. \mathbf{if} \ (\mathbf{isZero} \ \boxed{n}) \ \mathbf{1} \ (\mathbf{mult} \ \boxed{n} \ (\boxed{r} \ (\mathbf{pred} \ \boxed{n}))))}_{\mathbf{fac}' } \mathbf{fac} \ n$$

3. Правые аргументы совпадают, остаётся $\mathbf{fac} =_{\beta} \mathbf{fac}' \ \mathbf{fac}$: искомое — неподвижная точка $\mathbf{fac}'$, а её даёт комбинатор, так как $\underbrace{\mathbf{Y} \ F}_X =_{\beta} F \ (\underbrace{\mathbf{Y} \ F}_X)$:

$$\mathbf{fac} := \mathbf{Y} \ \mathbf{fac}' \equiv \mathbf{Y} \ (\lambda r \ n. \mathbf{if} \ (\mathbf{isZero} \ n) \ \mathbf{1} \ (\mathbf{mult} \ n \ (r \ (\mathbf{pred} \ n))))$$

- Δ Свойство комбинатора неподвижной точки $\mathbf{Y} \ F =_{\beta} F \ (\mathbf{Y} \ F)$ — единственное, что нужно знать про $\mathbf{Y}$, чтобы им пользоваться

Как работает Υ : fac 3

Раскрыли имя **fac**. Дальше единственный инструмент — свойство $\mathbf{Y} \ F =_{\beta} F \ (\mathbf{Y} \ F)$.

$\Upsilon \text{ fac}' \ 3$

ЦЕПОЧКА

$\text{fac } 3 \rightsquigarrow_{\text{fac}} \mathbf{Y} \text{ fac}' \ 3$

Как работает Y: fac 3

Свойство **Y**: наружу вышла копия **fac'**, а внутри остался ещё один **Y fac'** — запас на следующую итерацию.

fac' (Y fac') 3

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

Как работает Y : $\text{fac } 3$

Два β -шага: r стало копией $Y \text{ fac}'$, n стало 3 . Перед нами тело fac' , в нём никакой рекурсии.

```
if (isZero 3) 1 (mult 3 (Y fac' (pred 3)))
```

ЦЕПОЧКА

$\text{fac } 3 \rightsquigarrow_{\text{fac}} Y \text{ fac}' 3$

$=_{\beta} \text{fac}' (Y \text{ fac}') 3$ — свойство Y

$\rightarrow \text{if (isZero 3) 1}$
 $\quad (\text{mult 3 (Y fac' (pred 3))})$ — два β -шага

Как работает Y: fac 3

isZero 3 ложно, **if** выбрал ветку **else**, **pred 3** $\rightarrow$ **2**. Снаружи остался **mult 3**, внутри — тот же кадр, что в начале.

mult 3 (Y fac' 2)

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

$\rightarrow$ **if (isZero 3) 1**

(mult 3 (Y fac' (pred 3))) — два β -шага

$\rightarrow$ **mult 3 (Y fac' 2)** — законы **isZero**, **if**, **pred**

Как работает Y: fac 3

Вторая итерация: снова свойство **Y**, снова копия **fac'** наружу.

mult 3 (fac' (Y fac') 2)

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

$\rightarrow$ **if (isZero 3) 1**

(**mult 3 (Y fac' (pred 3))**) — два β -шага

$\rightarrow$ **mult 3 (Y fac' 2)** — законы **isZero, if, pred**

$=_{\beta}$ **mult 3 (fac' (Y fac') 2)** — свойство **Y**

Как работает Y: fac 3

И снова β -шаги и условие. Каждая итерация оставляет снаружи один **mult**.

mult 3 (mult 2 (Y fac' 1))

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

$\rightarrow$ **if (isZero 3) 1**

(mult 3 (Y fac' (pred 3))) — два β -шага

$\rightarrow$ **mult 3 (Y fac' 2)** — законы **isZero, if, pred**

$=_{\beta}$ **mult 3 (fac' (Y fac') 2)** — свойство **Y**

$\rightarrow$ **mult 3 (mult 2 (Y fac' 1))** — как выше

Как работает Y: fac 3

Третья итерация проходит так же: копия, β -шаги, условие ложно, снаружи ещё один **mult**.

mult 3 (mult 2 (mult 1 (Y fac' 0)))

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

$\rightarrow$ **if (isZero 3) 1**

(mult 3 (Y fac' (pred 3))) — два β -шага

$\rightarrow$ **mult 3 (Y fac' 2)** — законы **isZero, if, pred**

$=_{\beta}$ **mult 3 (fac' (Y fac') 2)** — свойство **Y**

$\rightarrow$ **mult 3 (mult 2 (Y fac' 1))** — как выше

$\rightarrow$ **mult 3 (mult 2 (mult 1 (Y fac' 0)))** —
итерация 3

Как работает Y: fac 3

Четвёртая итерация: копия, β -шаги, и на этот раз **isZero 0** истинно.

```
mult 3 (mult 2 (mult 1 (
if (isZero 0) 1 (mult 0 (Y fac' (pred 0))))))
```

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство Y

$\rightarrow$ **if (isZero 3) 1**

(mult 3 (Y fac' (pred 3))) — два β -шага

$\rightarrow$ **mult 3 (Y fac' 2)** — законы **isZero**, **if**, **pred**

$=_{\beta}$ **mult 3 (fac' (Y fac') 2)** — свойство Y

$\rightarrow$ **mult 3 (mult 2 (Y fac' 1))** — как выше

$\rightarrow$ **mult 3 (mult 2 (mult 1 (Y fac' 0)))** — итерация 3

$\rightarrow$ **mult 3 (mult 2 (mult 1 (if (isZero 0) 1
(mult 0 (Y fac' (pred 0)))))** — итерация 4

Как работает Y: fac 3

if вернул **1**, а ветка с **Y fac' (pred 0)** выброшена, так и не раскрывшись. **Y** разворачивался ровно столько раз, сколько потребовало условие.

mult 3 (mult 2 (mult 1 1)) → 6

ЦЕПОЧКА

fac 3 $\rightsquigarrow_{\text{fac}}$ **Y fac' 3**

$=_{\beta}$ **fac' (Y fac') 3** — свойство **Y**

→ **if (isZero 3) 1**

(mult 3 (Y fac' (pred 3))) — два β -шага

→ **mult 3 (Y fac' 2)** — законы **isZero, if, pred**

$=_{\beta}$ **mult 3 (fac' (Y fac') 2)** — свойство **Y**

→ **mult 3 (mult 2 (Y fac' 1))** — как выше

→ **mult 3 (mult 2 (mult 1 (Y fac' 0)))** —
итерация 3

→ **mult 3 (mult 2 (mult 1 (if (isZero 0) 1**
 (mult 0 (Y fac' (pred 0)))))) — итерация 4

→ **mult 3 (mult 2 (mult 1 1)) → 6** — закон **if**

Рекурсивное уравнение: fib

✎ Реализуйте **fib** n — n -е число Фибоначчи: запишите рекурсивное уравнение и решите его через **Y**. Считайте, что **le**, **plus** и **minus** уже есть.

⚡ **Решение.**

1. Уравнение: $\text{fib } n =_{\beta} \text{if } (\text{le } n \text{ 2}) \text{ 1 } (\text{plus } (\text{fib } (\text{minus } n \text{ 1})) (\text{fib } (\text{minus } n \text{ 2})))$
2. Абстрагируем правую часть по имени и аргументу: $\text{fib}' := \lambda r n. \text{if } (\text{le } n \text{ 2}) \text{ 1 } (\text{plus } (r (\text{minus } n \text{ 1})) (r (\text{minus } n \text{ 2})))$, и уравнение стало $\text{fib} =_{\beta} \text{fib}' \text{ fib}$
3. Решение: $\text{fib} := \mathbf{Y} \text{ fib}'$

✎ Найдите F такой, что $F =_{\beta} M F$ для произвольного M .

⚡ **Решение.**

Уравнение уже имеет вид « F — неподвижная точка M », абстрагировать нечего: $F := \mathbf{Y} M$

Кодирование данных

Варианты

Значения-варианты

Одна сущность бывает нескольких видов, и обрабатывать их нужно разным кодом: пользователь авторизован или нет, вершина выражения — константа или сложение.

```
def eval(node):  
    if is_const(node):  
        return node.value  # у константы должно быть значение  
    elif is_plus(node):  
        return eval(node.left) + eval(node.right)  # у плюса — операнды
```

API вариантов

- Два конструктора: «левый» **inl** и «правый» **inr**, каждый принимает одно значение
- Элиминатор **either** выполняет одну из двух переданных ему функций в зависимости от варианта

Законы вариантов

1. **either** F G (**inl** N) $=_{\beta}$ F N
2. **either** F G (**inr** M) $=_{\beta}$ G M

Закодируйте варианты

Законы вариантов $\text{either } F \ G \ (\text{inl } N) =_{\beta} F \ N,$ $\text{either } F \ G \ (\text{inr } M) =_{\beta} G \ M$

✎ Закодируйте варианты: **inl**, **inr** и **either**, удовлетворяющие законам. Способов не меньше двух.

✂ *Решение.*

Через пары с тегом: булев тег говорит, какой это вариант, а **either** выбирает код по тегу

$\text{inl} := \lambda x. \text{pair } \text{true } x$ $\text{inr} := \lambda y. \text{pair } \text{false } y$
 $\text{either} := \lambda f \ g \ v. \text{if } (\text{fst } v) (f (\text{snd } v)) (g (\text{snd } v))$

✂ *Решение.*

По Чёрчу: значение само принимает оба кода и применяет к себе нужный, элиминатору остаётся передать их

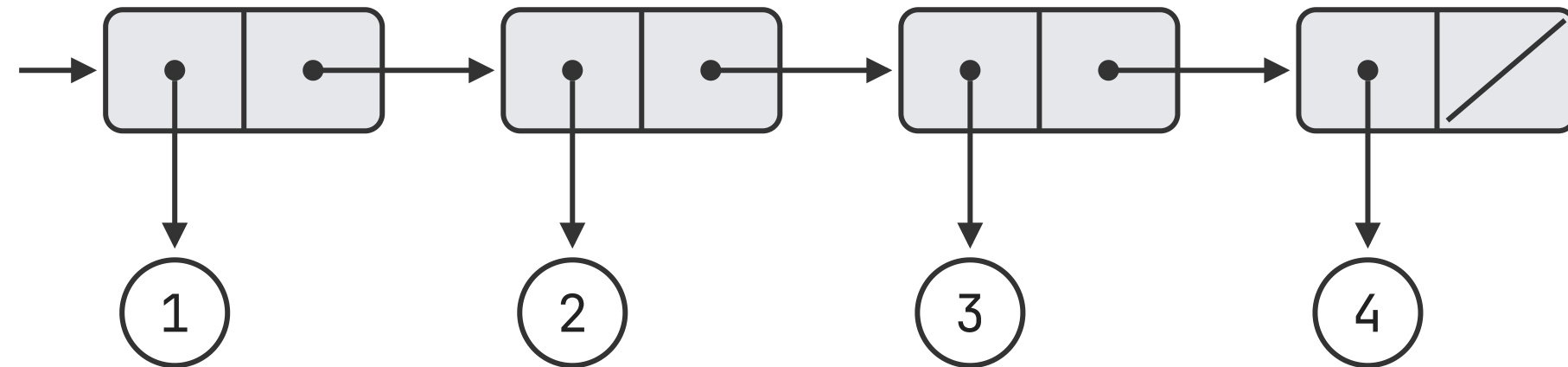
$\text{inl} := \lambda x. \lambda f \ g. f \ x,$ $\text{inr} := \lambda y. \lambda f \ g. g \ y,$ $\text{either} := \lambda f \ g \ v. v \ f \ g$

Списки

Списки

Иногда требуется хранить много сущностей одновременно, и при этом:

- Уметь добавить в коллекцию новую сущность
- Найти сущность в коллекции по индексу



Моделируем список

API списков

- **nil** — пустой список
- **cons** x t — список из элемента x в голове и списка-хвоста t
- e.g. $[1, 2, 3] \equiv \text{cons } 1 (\text{cons } 2 (\text{cons } 3 \text{ nil}))$
- Элиминатор, аккумулирующий элементы: **fold**
- e.g. $\text{fold } [1, 2, 3] F Z =_{\beta} F \ 1 \ (F \ 2 \ (F \ 3 \ Z))$

Законы списков

1. $\text{fold nil } F Z =_{\beta} Z$
2. $\text{fold } (\text{cons } N M) F Z =_{\beta} F \ N \ (\text{fold } M F Z)$

Закодируйте списки

✎ Закодируйте списки: **nil**, **cons** и **fold**, удовлетворяющие законам. Способов снова не меньше двух: через варианты и пары или по Чёрчу, по образцу чисел.

⚡ *Решение.*

Через варианты: список либо пуст, либо пара «голова, хвост»; **fold** рекурсивен, нужен **Y**

nil := **inl** 0

cons := $\lambda x t. \mathbf{inr} (\mathbf{pair} \ x \ t)$

fold := $\mathbf{Y} \ (\lambda r \ x s \ f \ z. \mathbf{either} \ \underbrace{(\mathbf{K} \ z)}_{\text{для inl}} \ \underbrace{(\lambda p. f \ (\mathbf{fst} \ p) \ (r \ (\mathbf{snd} \ p) \ f \ z))}_{\text{для inr: голова и свёртка хвоста}} \ x s)$

⚡ *Решение.*

По Чёрчу: как число применяет шаг k раз, список применяет шаг к каждому элементу; **fold** снова **I**

nil := $\lambda s \ z. z$, **cons** := $\lambda x \ t. \lambda s \ z. s \ x \ (t \ s \ z)$, **fold** := $\lambda x s \ f \ z. x s \ f \ z$

Как работает fold на вариантах: [1, 2]

Список через варианты: $[1, 2] \equiv \text{inr} (\text{pair } 1 [2])$, где $[2] \equiv \text{inr} (\text{pair } 2 [])$ и $[] \equiv \text{inl } 0$. Считаем **fold** законами **either** и пар.

`fold [1, 2] f z`

ЦЕПОЧКА

fold [1, 2] f z

Как работает fold на вариантах: [1, 2]

Раскрыли **fold** по свойству **Y** и сделали β -шаги: **either** с двумя ветками смотрит на xs , а это **inr**-вершина.

```
either (K z) ( $\lambda p.$  ...) (inr (pair 1 [2]))
```

ЦЕПОЧКА

```
fold [1, 2] f z
```

```
 $\rightarrow$  either (K z) ( $\lambda p.$  ...)
```

```
  (inr (pair 1 [2])) – раскрыли fold
```

Как работает fold на вариантах: [1, 2]

Закон **either** для **inr** выбрал вторую ветку и отдал ей пару. Вот теперь её тело важно: голова и свёртка хвоста.

```
(λp. f (fst p) (fold (snd p) f z)) (pair 1 [2])
```

ЦЕПОЧКА

fold [1, 2] *f* *z*

→ **either** (**K** *z*) (λ*p*. ...)

(**inr** (**pair** 1 [2])) — раскрыли **fold**

→ (λ*p*. ...) (**pair** 1 [2]) — закон **either** для **inr**

Как работает fold на вариантах: [1, 2]

Законы пар достали голову и хвост. Снаружи остался f 1, внутри — тот же **fold** на хвосте.

$f\ 1\ (\text{fold}\ [2]\ f\ z)$

ЦЕПОЧКА

fold [1, 2] $f\ z$

→ **either** (**K** z) ($\lambda p. \dots$)

(**inr** (**pair** 1 [2])) — раскрыли **fold**

→ ($\lambda p. \dots$) (**pair** 1 [2]) — закон **either** для **inr**

→ $f\ 1\ (\text{fold}\ [2]\ f\ z)$ — **законы пар**

Как работает fold на вариантах: [1, 2]

Вторая вершина проходит так же: ещё один вызов f снаружи.

$f\ 1\ (f\ 2\ (\text{fold}\ []\ f\ z))$

ЦЕПОЧКА

fold [1, 2] $f\ z$

→ **either** (**K** z) ($\lambda p. \dots$)

(**inr** (**pair** 1 [2])) — раскрыли **fold**

→ ($\lambda p. \dots$) (**pair** 1 [2]) — закон **either** для **inr**

→ $f\ 1\ (\text{fold}\ [2]\ f\ z)$ — законы пар

→ $f\ 1\ (f\ 2\ (\text{fold}\ []\ f\ z))$ — вторая вершина

Как работает fold на вариантах: [1, 2]

Дошли до **inl**: закон **either** для **inl** выбирает **K** z , и что бы ни лежало в конце, ответ z .

$f\ 1\ (f\ 2\ (\text{either}\ (K\ z)\ (\lambda p. \dots)\ (\text{inl}\ 0)))$

ЦЕПОЧКА

fold [1, 2] $f\ z$

→ **either** (**K** z) ($\lambda p. \dots$)

(**inr** (**pair** 1 [2])) — раскрыли **fold**

→ ($\lambda p. \dots$) (**pair** 1 [2]) — закон **either** для **inr**

→ $f\ 1\ (\text{fold}\ [2]\ f\ z)$ — законы пар

→ $f\ 1\ (f\ 2\ (\text{fold}\ []\ f\ z))$ — вторая вершина

→ $f\ 1\ (f\ 2\ (\text{either}\ (K\ z)\ (\lambda p. \dots)\ (\text{inl}\ 0)))$

Как работает fold на вариантах: [1, 2]

Ровно законы **fold**: каждый **cons** дал вызов f , **nil** дал z . Список через варианты обходится рекурсией через **Y**; список Чёрча делает то же сам.

$f\ 1\ (f\ 2\ (z))$

ЦЕПОЧКА

fold [1, 2] $f\ z$

→ **either** (**K** z) ($\lambda p. \dots$)

(**inr** (**pair** 1 [2])) — раскрыли **fold**

→ ($\lambda p. \dots$) (**pair** 1 [2]) — закон **either** для **inr**

→ $f\ 1\ (\mathbf{fold}\ [2]\ f\ z)$ — законы пар

→ $f\ 1\ (f\ 2\ (\mathbf{fold}\ []\ f\ z))$ — вторая вершина

→ $f\ 1\ (f\ 2\ (\mathbf{either}\ (\mathbf{K}\ z)\ (\lambda p. \dots)\ (\mathbf{inl}\ 0)))$

→ $f\ 1\ (f\ 2\ (\mathbf{K}\ z\ 0)) \rightarrow f\ 1\ (f\ 2\ z)$ — закон **either** для **inl**

Список и вариант вместе

✎ Реализуйте **safeHead**: голову списка как вариант, чтобы пустой список не был ошибкой, а возвращал значение по умолчанию. Спецификация:

- $\text{safeHead } D \text{ nil} =_{\beta} \text{inl } D$
- $\text{safeHead } D (\text{cons } N M) =_{\beta} \text{inr } N.$

⚡ *Решение.*

$$\text{safeHead} := \lambda d \, xs. \text{fold } xs \underbrace{(\lambda x \, t. \text{inr } x)}_{\text{для cons}} \underbrace{(\text{inl } d)}_{\text{для nil}}$$

Шаг игнорирует результат для хвоста r и оборачивает голову; для пустого списка **fold** возвращает начальное значение **inl** d . Работает с любым из двух кодирований — мы пользовались только законами.

Наш язык на сегодня

Две пары назад в языке были только переменные, абстракция и аппликация. Всё ниже собрано из них.

комбинаторы	лекция 1	I K K_* S B
пары	П1	pair fst snd swap
логика	П1	true false if not and or
числа	П1, hw01	0 suc natElim plus mult isZero isEven
циклы	П2	состояние-пара через natElim: fac pred
рекурсия	лекция 2, П2	Y fac fib
варианты	П2	inl inr either
списки	П2	nil cons fold safeHead

Сегодня мы узнали много нового

- Число итераций известно заранее — рекурсия не нужна
- Число шагов решается по ходу — рекурсивное уравнение: абстрагировать по имени и аргументу, взять неподвижную точку через Y
- Проверяем законами, «большими шагами», а не раскрытием до λ
- Переизобрели варианты (суммы) и списки (коллекции)

Материалы

Выбор инструмента: цикл с фиксированным числом итераций vs рекурсия

- Цикл с фиксированным числом итераций
 - Задаёт процесс из конечного, заранее заданного, количества шагов
 - Аналог цикла `for in range` в современных языках
- Рекурсия
 - Количество шагов определяется в процессе (и никак иначе: halting problem)
 - Завершение процесса определяется выполнением произвольного условия (базы рекурсии)
 - В современных языках выражается рекурсией¹ и циклом `while condition: ...`
- Всегда нужно выбирать *наименее могущественный* из подходящих инструментов
 - Проще для рассуждений и доказательства свойств
 - Меньше пространство для ошибок
 - Скорее всего, код будет эффективнее исполняться
 - В частности, какие функции выражаются таким циклом, а какие — нет, рассказывает теория вычислимости (это и есть примитивно-рекурсивные функции)

1. Хвостовая рекурсия полностью эквивалентна обычной, хотя и похожа на цикл (см. [trampolining](#)).

Система рекурсивных уравнений

Взаимная рекурсия: две функции ссылаются друг на друга, а **Y** ищет одну неподвижную точку.

$$\begin{cases} \text{isEven } n =_{\beta} \text{ if } (\text{isZero } n) \text{ true } (\text{isOdd } (\text{pred } n)) \\ \text{isOdd } n =_{\beta} \text{ if } (\text{isZero } n) \text{ false } (\text{isEven } (\text{pred } n)) \end{cases}$$

Две неизвестные упаковываем в одну пару $p = \text{pair isEven isOdd}$ и решаем уравнение на неё как обычно:

$$P := \mathbf{Y} \left(\lambda p. \underbrace{\text{pair } (\lambda n. \text{ if } (\text{isZero } n) \text{ true } (\text{snd } p (\text{pred } n)))}_{\text{isEven, где snd } p \text{ — это isOdd}} \right. \\ \left. \underbrace{(\lambda n. \text{ if } (\text{isZero } n) \text{ false } (\text{fst } p (\text{pred } n))))}_{\text{isOdd, где fst } p \text{ — это isEven}} \right)$$

Тогда **isEven** := **fst** P , **isOdd** := **snd** P . Пара третий раз за пару оказывается «состоянием»: в разминке, в **pred** и здесь.

Объектно-ориентированная запись вариантов по Чёрчу

Как и у пар в П1: интерфейс с одним методом-элиминатором, по анонимному классу на конструктор. Захваченные аргументы конструктора — это то, что $\lambda f g. f x$ замыкает.

```
interface Variant<A, B> {
    fun <R> either(f: (A) -> R, g: (B) -> R): R
}

fun <A, B> inl(x: A): Variant<A, B> = object : Variant<A, B> {
    override fun <R> either(f: (A) -> R, g: (B) -> R): R = f(x)
}

fun <A, B> inr(y: B): Variant<A, B> = object : Variant<A, B> {
    override fun <R> either(f: (A) -> R, g: (B) -> R): R = g(y)
}
```

Тегированное кодирование в ООП — это поле `tag` и `switch` по нему в каждом месте использования. Кодирование по Чёрчу переносит выбор ветки в диспетчеризацию метода — паттерн «Visitor».

Объектно-ориентированная запись списков Чёрча

```
interface List<A> {  
    fun <R> fold(s: (A, R) -> R, z: R): R  
}  
  
fun <A> nil(): List<A> = object : List<A> {  
    override fun <R> fold(s: (A, R) -> R, z: R): R = z  
}  
  
fun <A> cons(x: A, t: List<A>): List<A> = object : List<A> {  
    override fun <R> fold(s: (A, R) -> R, z: R): R = s(x, t.fold(s, z))  
}  
  
fun sum(xs: List<Int>): Int = xs.fold({ x, r -> x + r }, 0)  
  
fun <A> concat(xs: List<A>, ys: List<A>): List<A> = xs.fold({ x, r -> cons(x, r) }, ys)  
  
fun <A> safeHead(d: A, xs: List<A>): Either<A, A> = xs.fold({ x, _ -> inr(x) }, inl(d))
```

Списки в Си

```
struct Node {  
    int value;  
    Node * tail;  
};  
  
Node * nil = NULL;  
  
Node * cons(int value, Node * tail) {  
    return new Node { value, tail };  
}
```

Что посмотреть в транспорте

- [Статья: Прimitивно-рекурсивные функции и функция Аккермана](#)

Серьёзные материалы

- Klop, J. W. «New fixed point combinators from old.» Reflections on Type Theory, λ -Calculus, and the Mind. Essays dedicated to Henk Barendregt on the occasion of his 60th birthday, Radboud University Nijmegen (2007): 197–211